model_training: small training examples

What this folder shows

The paper's results use the provided trained models (the published TCHES20 models and the released Transformer checkpoints), so nothing in the reproduction trains a model. This folder shows, at a small scale, how each kind of model in the paper is trained: `train_mlp_example.py` (a plain network, scikit-learn), `train_cnn_example.py` (a convolutional network, PyTorch) and `train_transformer_example.py` (a small Transformer, PyTorch). The models they save cannot be used by the certificate scripts, which expect the provided model files.

Before you start

`requirements.txt` for the plain network, `requirements-ml.txt` for the other two, and some traces: the TCHES20 download works with `--dataset ascad_desync_0`, `ascad_desync_50`, `ascad_desync_100`, `aes_rd` or `dpav4`. The first two examples run on a CPU in minutes. The Transformer example uses a GPU if one is present; on a CPU, lower the number of epochs or shorten the window.

Run

```
T=datasets/raw/tches20/TCHES20V3_CNN_SCA
python -m model_training.train_mlp_example --dataset ascad_desync_0 --data-dir $T/datasets --tches20-src-dir $T/src
```

```
$ T=datasets/raw/tches20/TCHES20V3_CNN_SCA; python -m model_training.train_mlp_example --dataset ascad_desyn
    c_0 --data-dir $T/datasets --tches20-src-dir $T/src
[... earlier output omitted; key lines kept ...]
[...]
Iteration 8, loss = 4.99303127
Validation score: 0.006667
Iteration 9, loss = 4.83640457
Validation score: 0.004167
Iteration 10, loss = 4.65057517
Validation score: 0.004167
Iteration 11, loss = 4.46258535
Validation score: 0.005833
Iteration 12, loss = 4.26492901
Validation score: 0.003333
Iteration 13, loss = 4.06795095
Validation score: 0.004167
Iteration 14, loss = 3.86623797
Validation score: 0.006667
Iteration 15, loss = 3.65781447
Validation score: 0.003333
Iteration 16, loss = 3.44810520
Validation score: 0.005833
Iteration 17, loss = 3.24248798
Validation score: 0.006667
Iteration 18, loss = 3.06100135
Validation score: 0.004167
Iteration 19, loss = 2.88744986
Validation score: 0.000833
Validation score did not improve more than tol=0.000100 for 10 consecutive epochs. Stopping.
```

```
validation_accuracy=0.002500
holdout_accuracy=0.002750
wrote checkpoints/model_training/ascad_desync_0/mlp

100%|██████████| 10000/10000 [00:02<00:00, 4433.71it/s]
[exit code 0, 8 s]
```

```
python -m model_training.train_cnn_example --dataset ascad_desync_0 --data-dir $T/datasets --tches20-src-dir $T/src --epochs 3
```

```
$ T=datasets/raw/tches20/TCHES20V3_CNN_SCA; python -m model_training.train_cnn_example --dataset ascad_desyn
    c_0 --data-dir $T/datasets --tches20-src-dir $T/src --epochs 3
Creating all target labels for the attack traces:
  Loaded via TCHES20 dataLoaders: 50000 profiling traces, 700 features
epoch=1 loss=5.546170 val_accuracy=0.003500
epoch=2 loss=5.541369 val_accuracy=0.003750
epoch=3 loss=5.539213 val_accuracy=0.003750
holdout_accuracy=0.002750
wrote checkpoints/model_training/ascad_desync_0/cnn

100%|██████████| 9960/10000 [00:02<00:00, 4276.06it/s]
100%|██████████| 10000/10000 [00:02<00:00, 4301.33it/s]
[exit code 0, 8 s]
```

```
$ ls -R checkpoints/model_training && \
    cat checkpoints/model_training/ascad_desync_0/mlp/metadata.json
checkpoints/model_training:
ascad_desync_0

checkpoints/model_training/ascad_desync_0:
cnn
mlp

checkpoints/model_training/ascad_desync_0/cnn:
metadata.json
model.pt
scaler.pkl

[... 11 lines omitted ...]
    "hidden_layers": "512,256",
    "holdout_size": 0.2,
    "learning_rate": 0.001,
    "max_samples": 20000,
    "out_dir": "checkpoints/model_training",
    "seed": 1,
    "stratify": false,
    "tches20_src_dir": "datasets/raw/tches20/TCHES20V3_CNN_SCA/src",
    "train_size": 0.6,
    "val_size": 0.2,
    "window_end": null,
    "window_start": null
  },
  "metrics": {
    "holdout_accuracy": 0.00275,
    "val_accuracy": 0.0025
  }
}
[exit code 0, 0 s]
```

What to look for

Each example prints the sizes of the training, validation and test parts, the training progress, and the accuracy on the test part. It saves the model and a `metadata.json` with the settings and results.